Disjoint Set Union (DSU)

Company: Amazon
Round: Offline
Year: 2026
Difficulty: Medium
Topic: 2D Geometry, Arrays, Optimization
Source: https://www.desiqna.in/19291/amazon-sde-1-recent-interview-experiences-2026-set-67

Problem Description

DSU is used to manage different groups or sets. It can quickly tell whether two elements belong to the same group and can also merge two groups.

Input Format

1. The first line contains two integers N and Q:

- N → number of elements.
- Q → number of operations.

2. The next Q lines contain an operation:

- 1 a b → Union the sets containing a and b.
- 2 a b → Find/Check whether a and b belong to the same set.

Output Format

For each operation of type 2 a b, print:

- YES if a and b belong to the same set.
- NO if a and b belong to different sets.

Each answer should be printed on a separate line.

Sample Input

5 4
1 1 2
1 2 3
2 1 3
2 1 5

Sample Output

YES
NO

Explanation

Initially:

{1} {2} {3} {4} {5}

After 1 1 2:

{1,2} {3} {4} {5}

After 1 2 3:

{1,2,3} {4} {5}

So 1 and 3 are in the same set → YES.

But 1 and 5 are in different sets → NO.

Approach to Solve This Problem:

- Initially, every element is in its own set.
- Use find() to find the parent of an element.
- Use union() to join two sets.
- Path compression and union by rank/size make the operations fast.

Java Solution:

```java
import java.util.*;
public class Main {
    static int[] parent;
    static int[] rank;
    static int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }
        return parent[x];
    }
    static void union(int a, int b) {
        int rootA = find(a);
        int rootB = find(b);
        if (rootA == rootB) {
            return;
        }
        if (rank[rootA] < rank[rootB]) {
            parent[rootA] = rootB;
        } else if (rank[rootA] > rank[rootB]) {
            parent[rootB] = rootA;
        } else {
            parent[rootB] = rootA;
```

```
        } else {
            parent[rootB] = rootA;
            rank[rootA]++;
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int q = sc.nextInt();
        parent = new int[n + 1];
        rank = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            parent[i] = i;
        }
        for (int i = 0; i < q; i++) {
            int type = sc.nextInt();
            int a = sc.nextInt();
            int b = sc.nextInt();
            if (type == 1) {
                union(a, b);
            } else if (type == 2) {
                if (find(a) == find(b)) {
                    System.out.println("YES");
                } else {
                    System.out.println("NO");
                }
```

Solution:

The main idea is to use a **HashSet** to quickly check whether a point exists.

1. Store all the given points in a HashSet.
2. Select any two points as possible **opposite corners** of a rectangle.
3. Ignore the pair if they have the same x or same y coordinate.
4. Calculate the positions of the other two corners.
5. Check whether both missing corners exist in the HashSet.
6. If they exist, calculate the rectangle's width and height.
7. Calculate the area and keep the **maximum area** found.

Complexity Analysis:

Time Complexity: O(α(N))

As per operation, which is approximately **O(1)** in practice.
Path compression and union by rank make find() and union() very efficient.

Space Complexity: O(N)

We store the parent and rank arrays for all N elements